FASTAPI RAG PROJECT

Technical Documentation & Implementation Guide

Backend	FastAPI + SQLAlchemy
Database	PostgreSQL + pgvector
Local AI	Ollama
Embeddings	nomic-embed-text · 768 dimensions
Answer model	llama3.2:3b (configured in the current project)

Purpose

This document explains the project’s Retrieval-Augmented Generation (RAG) workflow: loading documentation, splitting text into chunks, generating embeddings locally, storing vectors in PostgreSQL, retrieving relevant chunks, and generating an answer with source references.

Prepared as a practical project guide. Details marked as configuration-dependent should be checked against the current source code and environment.

1. Project Overview

Retrieval-Augmented Generation combines document retrieval with language-model generation. Instead of asking the model to answer only from its built-in knowledge, the application retrieves relevant project documentation and supplies that context to the answer model.

The current implementation has been tested with a Markdown README. The API returned an answer about FastAPI, PostgreSQL, and pgvector and included source references such as `README.md`. PDF loader tests also passed in the development environment.

1.1 Main components

Component	Responsibility	Current project detail
FastAPI	Exposes API endpoints	RAG chat route: POST /rag/chat
SQLAlchemy	Database sessions and ORM	DocumentChunk model
PostgreSQL	Stores chunk text and metadata	Database: fastapi_crud_db
pgvector	Stores/searches vector embeddings	Vector dimension: 768
Ollama	Runs models locally	nomic-embed-text for embeddings
Answer model	Writes response using retrieved context	llama3.2:3b configured in project
React + Vite	Frontend application	Can call the backend using VITE_API_URL
Docker Compose	Runs services together	db, backend, frontend services

1.2 RAG flow at a glance

Stage	What happens
Ingestion	Read a PDF or Markdown file and extract its text.
Chunking	Split extracted text into smaller pieces with chunk indexes.
Embedding	Convert each chunk into a numeric vector using nomic-embed-text.
Storage	Save source, index, text, metadata, and embedding in PostgreSQL.
Retrieval	Embed the user’s question and search for relevant document chunks.
Generation	Send the retrieved context and question to the local answer model.
Response	Return the answer and source references to the caller.

2. Architecture & Configuration

The development setup uses Docker Compose for the database and application services, while Ollama runs on the Windows host. When the backend runs inside Docker, it can reach the host machine's Ollama API using `host.docker.internal`. When a Python script runs directly in Windows, use `localhost` for Ollama.

2.1 Connection addresses

Caller	Service	Address / port
Windows Python script	Ollama	http://localhost:11434
Docker backend	Ollama on Windows host	http://host.docker.internal:11434
Windows database client	PostgreSQL in Docker	localhost:5433
Backend container	PostgreSQL service	db:5432
Browser / Swagger	FastAPI backend	http://localhost:8000
Browser development server	React + Vite	http://localhost:5173

2.2 Important environment settings

```
# Windows-only ingestion script; set this in the current PowerShell session:
$env:OLLAMA_BASE_URL = "http://localhost:11434"

# Docker backend should use Docker service DNS for PostgreSQL:
# postgresql+psycopg://<user>:<password>@db:5432/<database>

# Frontend example:
# VITE_API_URL=http://localhost:8000
```

Do not commit passwords, tokens, or other secrets to source control. Keep real credentials in local environment files and provide only placeholder values in .env.example. The backend's internal database address and a Windows-host script's database address are different because Docker port mapping is involved.

2.3 Model responsibilities

- **nomic-embed-text:** produces 768-dimensional vectors for document chunks and user queries.
- **llama3.2:3b:** generates natural-language answers from the retrieved context.
- **pgvector:** supports vector similarity search; it does not generate answers.
- **FastAPI:** coordinates the API request, retrieval, model call, and response.

3. Document Ingestion

The ingestion process reads supported documents, extracts text, chunks the text, generates embeddings, and saves records. The project currently supports PDF and Markdown extensions in the ingestion service.

3.1 DocumentChunk record

Field	Meaning
id	Primary-key identifier.
source	Filename/source label used in returned references.
chunk_index	Zero-based position of the chunk within the document.
text	Extracted text stored for retrieval.
metadata_json	Optional JSON metadata.
embedding	Nullable pgvector vector with 768 dimensions.

3.2 Conceptual ingestion sequence

Step	Operation	Expected result
1	Validate file extension	Accept PDF, .md, or .markdown.
2	Load file text	PDF or Markdown text is extracted.
3	Chunk text	A list of indexed text chunks is produced.
4	Generate embeddings	One 768-dimensional embedding per chunk.
5	Save chunk records	Source, index, text, and metadata are persisted.
6	Store embeddings	Vectors are stored on their matching chunk records.

3.3 Windows example: run a local ingestion script

```
# From the project root in PowerShell:
$env:OLLAMA_BASE_URL = "http://localhost:11434"
python .\ingest_rag_test.py
```

The exact document path is determined by the ingestion script. Before running it, confirm that the path exists and that the file contains extractable text. An empty PDF can produce no useful chunks even if it is a valid file.

3.4 PDF limitations

- A text-based PDF is usually extractable with a PDF text extraction library such as pypdf.
- A scanned/image-only PDF may need OCR; ordinary text extraction may return little or no text.
- Very large PDFs should be tested for chunk count, memory usage, and embedding latency.
- Validate that each generated embedding has 768 values before storing it.

4. Retrieval & Answer Generation

At query time, the application converts the user's question into an embedding, retrieves relevant chunks from PostgreSQL/pgvector, and provides the selected context to the answer model. The response includes source references so the caller can see which document chunks informed the answer.

4.1 Request lifecycle

Order	Action
1	Client sends a question to POST /rag/chat.
2	Authentication is checked if required by the route.
3	The query embedding is generated locally.
4	The retriever searches stored document vectors.
5	Retrieved text is assembled as context.
6	Ollama's answer model generates the response.
7	API returns an answer and source entries.

4.2 Example response shape

```
{
  "answer": "The project uses FastAPI, PostgreSQL, and pgvector.",
  "sources": [
    {
      "source": "README.md",
      "chunk_index": 0
    }
  ]
}
```

This is an illustrative response shape based on the working test flow; actual answer wording and source count depend on the question, stored documents, and retrieval results.

4.3 Test in Swagger UI

- Start the backend and open `http://localhost:8000/docs`.
- Find `POST /rag/chat` and inspect the request schema.
- Authorize with a valid JWT if the route requires authentication.
- Ask a question that is explicitly answered by an ingested document.
- Check both `answer` and `sources`.

If no relevant chunks exist, a fallback answer may be returned. A successful HTTP response alone does not prove that retrieval is correct; the answer and sources should be inspected.

5. Duplicate Prevention & Database Integrity

The project has been updated to prevent repeated ingestion of the same source/chunk index. The application-level repository checks for an existing (source, chunk_index) pair, and the PostgreSQL table now has a unique constraint named uq_document_chunks_source_index.

5.1 Why both layers matter

Layer	Protection	Limit
Application check	Skips existing records during ordinary ingestion.	A race condition can occur if simultaneous requests both check before either inserts.
Unique database constraint	Rejects duplicate (source, chunk_index) pairs at the database layer.	The application should handle uniqueness violations gracefully if concurrent inserts occur.

5.2 Current development database notes

- Database: `fastapi_crud_db`.
- A SQL dump was created as `fastapi_crud_db_backup.sql` before duplicate cleanup.
- One duplicate README.md row was removed after confirming both its text and embedding matched the retained row.
- Current inspection showed one row for README.md, chunk_index 0, and one row for README-rag-test.md, chunk_index 0.

The unique key is based on source filename and chunk index. If a file's contents change but the filename stays the same, this strategy does not automatically replace the old chunks. A future improvement is content hashing/versioning and a transaction that keeps chunk and embedding writes consistent.

6. Tests & Verification

The following test commands have been run successfully in the development environment.

```
# Full RAG test suite
python -m pytest app/rag/tests/ -q

# PDF/Markdown document loader tests
python -m pytest app/rag/tests/test_document_loaders.py -q
```

Recorded results

Test run	Recorded result
RAG test suite after duplicate-prevention updates	30 passed
Document loader tests	3 passed

Integration tests should use a dedicated test database. Do not point destructive test fixtures at the main project database. Confirm the test database URL before running integration tests.

7. Troubleshooting

Symptom	Likely cause	What to check
Connection refused on localhost:5432	Windows script using the container port instead of mapped host port.	Use localhost:5433 from Windows; use db:5432 inside Docker.
Ollama connection error	Wrong base URL or Ollama not running.	Windows script: localhost:11434; Docker backend: host.docker.internal:11434.
Embedding dimension error	Model/vector schema mismatch.	Confirm nomic-embed-text returns 768 values and column is Vector(768).
No relevant documentation found	No chunks ingested, query mismatch, or retrieval threshold.	Inspect document_chunks and source references.
Empty PDF yields no content	PDF has no text or is scanned.	Check file size and extracted text; OCR may be needed.
Duplicate constraint error	Existing duplicate source/index rows.	Inspect duplicates before adding or migrating the constraint.
Tests report no tests found	Incorrect test folder path.	Use app/rag/tests/ for this project.

8. Recommended Next Steps

- **Ingest a real PDF:** put a non-empty project documentation PDF in the docs folder and use the existing PDF loader.
- **Verify extracted text:** confirm that chunks are non-empty before generating embeddings.
- **Verify source references:** ask a question answered by the PDF and ensure the PDF filename appears in sources.
- **Handle concurrent ingestion:** catch unique-constraint violations and ensure the chunk/embedding write path is transaction-safe.
- **Support document updates:** consider content hashes or document versions so changed files can be re-ingested safely.
- **Connect the React UI:** add a chat form that calls the authenticated RAG endpoint and renders the answer with source references.
- **Operational hardening:** add upload size/type limits, error logging, and safeguards for untrusted document content.

9. Quick Reference

Purpose	Command / location
Run all RAG tests	python -m pytest app/rag/tests/ -q
Run loader tests	python -m pytest app/rag/tests/test_document_loaders.py -q
Swagger UI	http://localhost:8000/docs
Windows Ollama API	http://localhost:11434
Windows PostgreSQL connection	localhost:5433
Backend PostgreSQL connection	db:5432

Purpose	Command / location
Main database	fastapi_crud_db
RAG endpoint	POST /rag/chat

End of document

This guide reflects the project details and test results established during the current development session. Re-check configuration values against the source code before using this document as an operational runbook.